

Отчёт по лабораторной работе №5

Выполнила студентка НКАбд-02-25

Валерия Сергеевна Григорьева

Содержание

1	Цель работы	3
2	Порядок выполнения лабораторной работы	4
3	Задание для самостоятельной работы	16
4	Вывод	23

1 Цель работы

Цель данной работы — приобретение навыков работы в Midnight Commander. Освоение инструкций языка ассемблера `mov` и `int`.

2 Порядок выполнения лабораторной работы

В начале работы я открыла Midnight Commander с помощью команды `mc` (Рисунок 2.1).

```
vsgrigoreva@fedora:~$ mc
```

Рисунок 2.1: Команда `mc`

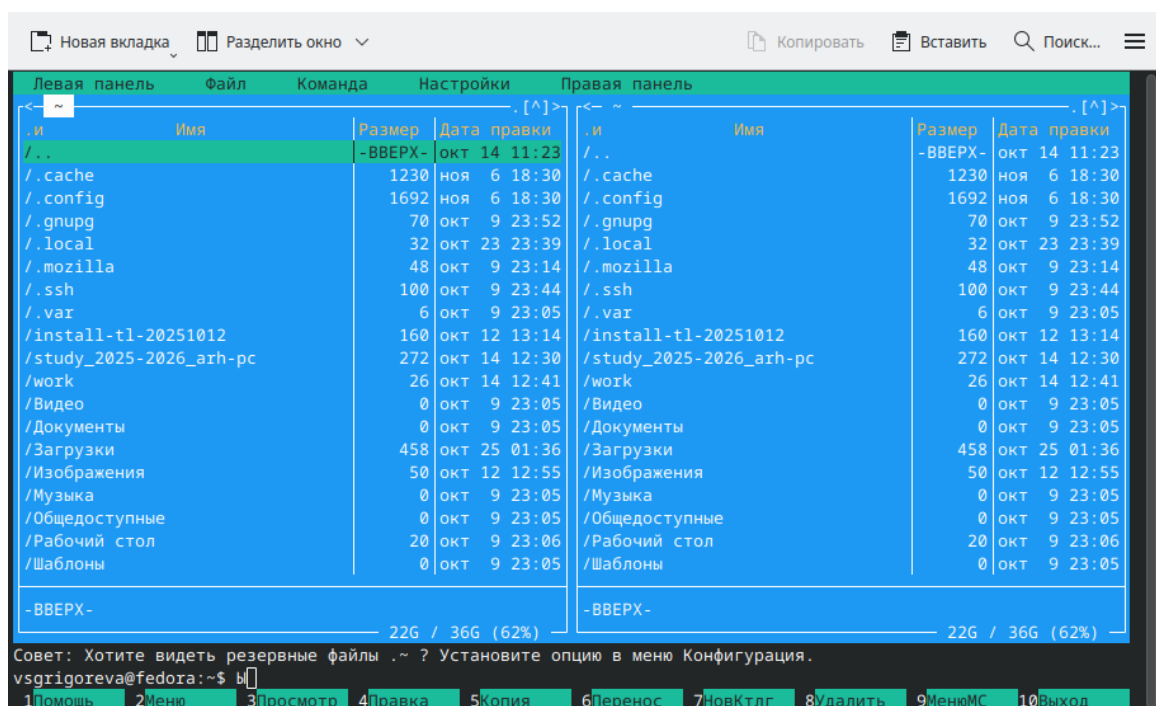


Рисунок 2.2: Midnight Commander

Далее с помощью стрелок на клавиатуре и клавиши Enter я перешла в каталог `~/work/arch-pc`, созданный при выполнении лабораторной работы №4 (Рисунок 2.3).

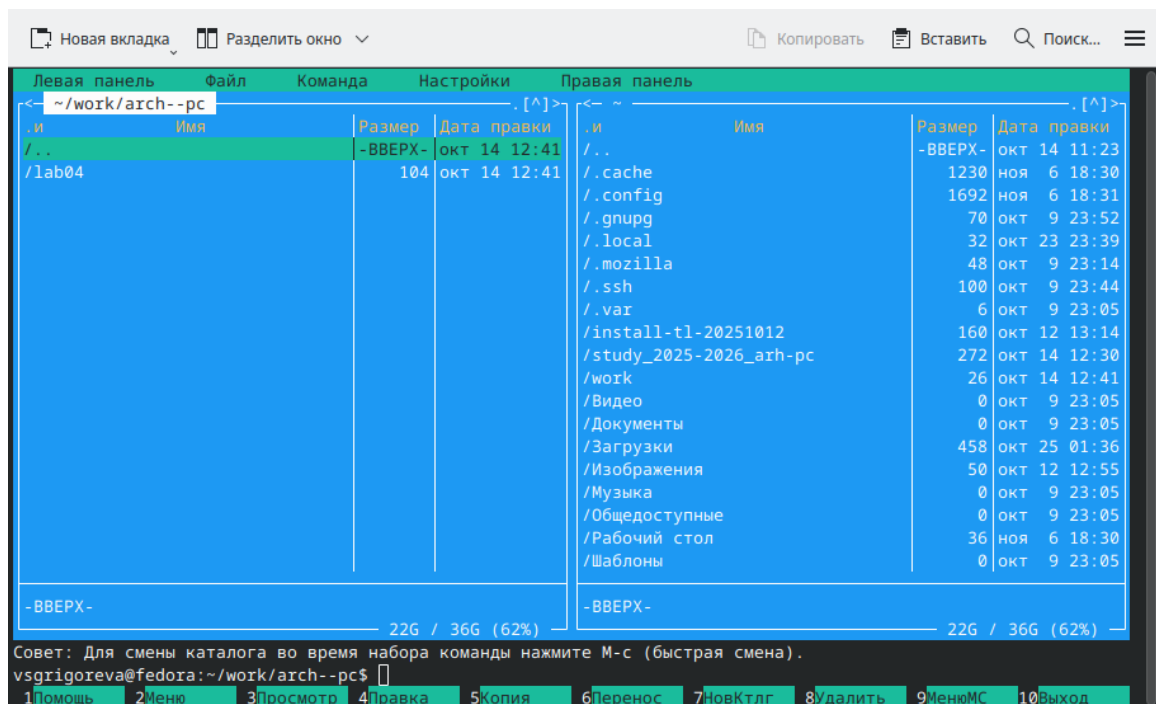


Рисунок 2.3: Каталог `~/work/arch-pc` в Midnight Commander

Затем я создала папку `lab05` с помощью клавиши F7 (Рисунок 2.4) и перешла в нее (Рисунок 2.5).

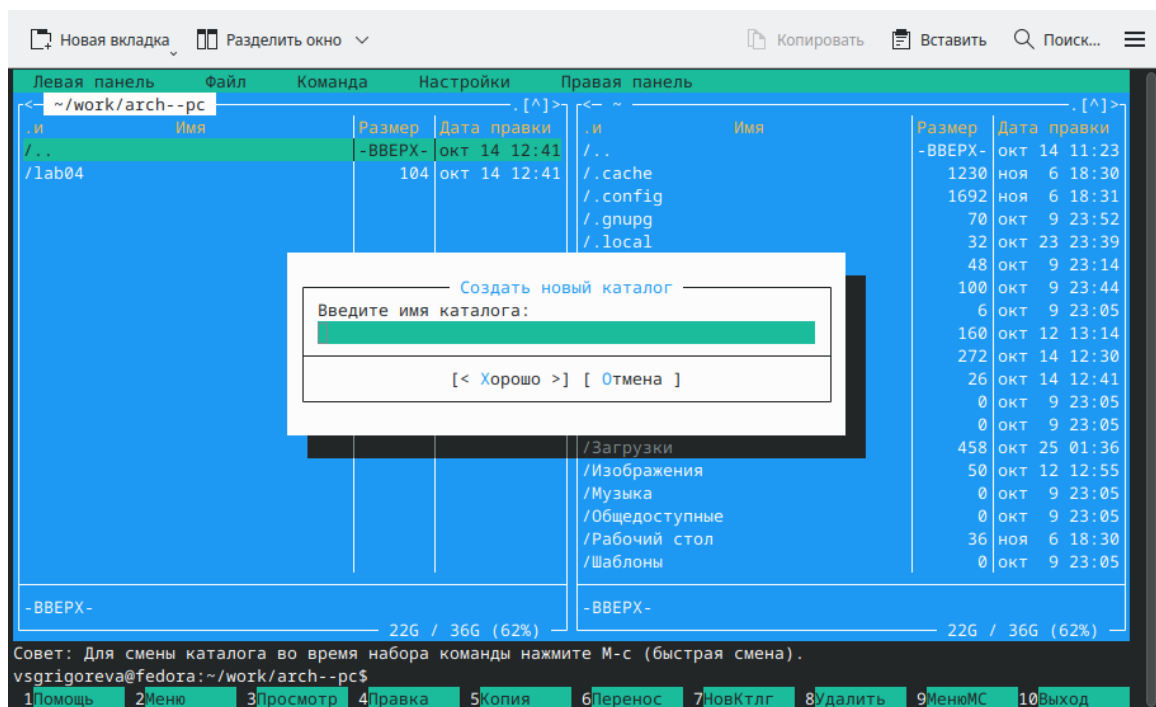


Рисунок 2.4: Создание каталога

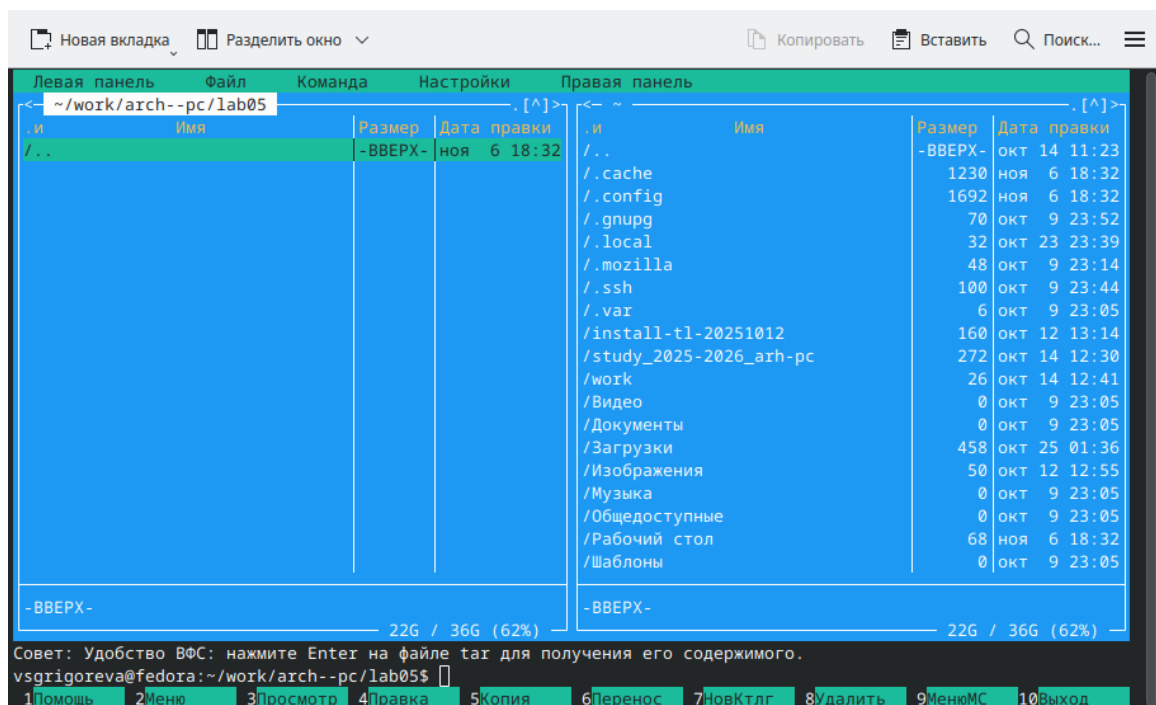


Рисунок 2.5: Каталог lab05

Затем, пользуясь командной строкой внизу страницы, я создала файл lab5-1.asm с помощью команды touch (Рисунок 2.6).

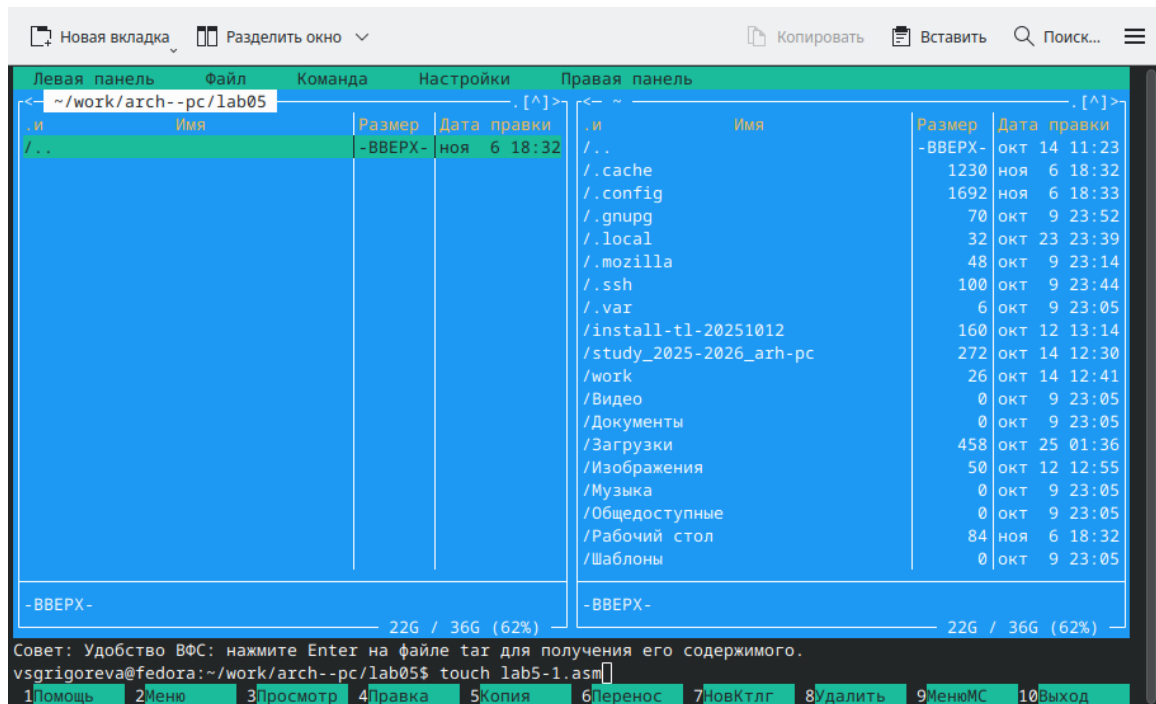


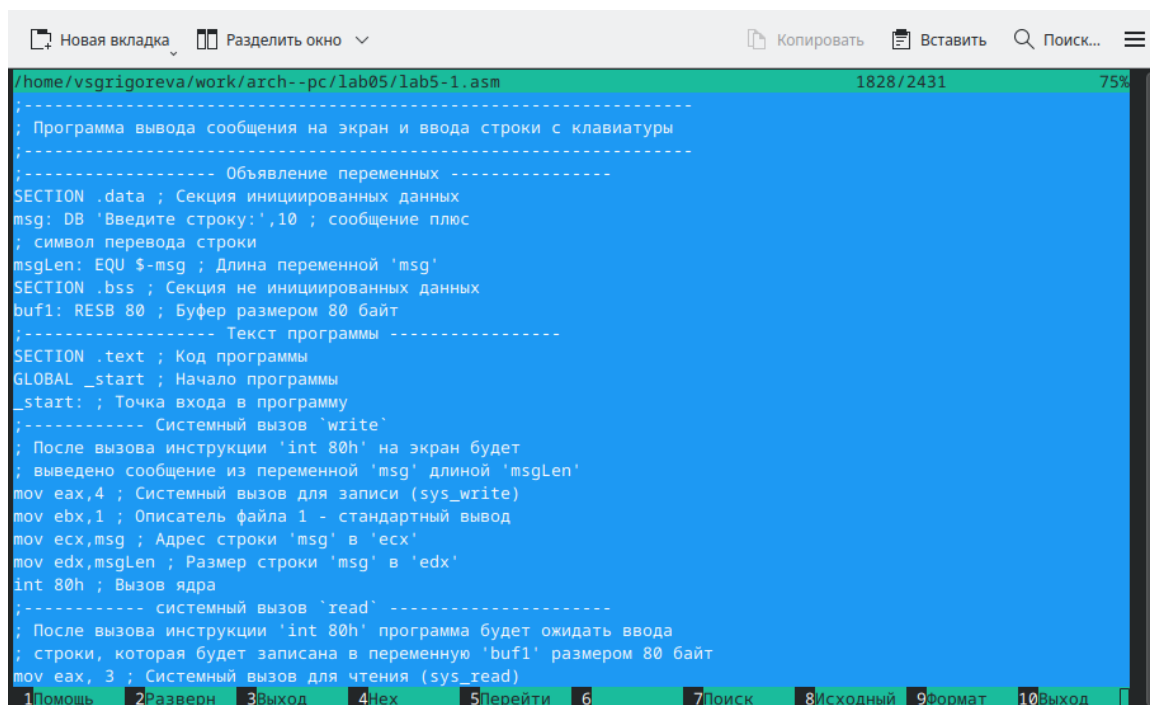
Рисунок 2.6: Создание файла lab5-1.asm

С помощью клавиши F4 я открыла файл lab5-1.asm для редактирования во встроенном редакторе. У меня используется редактор mcedit. Затем я ввела текст программы из листинга 5.1 (Рисунок 2.7).

```
lab5-1.asm [-M--] 31 L: [ 1+ 9 10/ 35] *(701 /2431b) 0056 0x038 [*][X]
;-----
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----
;----- Объявление переменных -----
SECTION .data ; Секция инициализированных данных
msg: DB "Введите строку:",10 ; сообщение плюс
; символ перевода строки
msgLen: EQU $-msg ; Длина переменной 'msg'
SECTION .bss ; Секция не инициализированных данных
buf1: RESB 80 ; Буфер размером 80 байт
;----- Текст программы -----
SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
;----- Системный вызов 'write' -----
; После вызова инструкции 'int 80h' на экран будет
; выведено сообщение из переменной 'msg' длиной 'msgLen'
mov eax,4 ; Системный вызов для записи (sys_write)
mov ebx,1 ; Описатель файла 1 - стандартный вывод
mov ecx,msg ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen ; Размер строки 'msg' в 'edx'
int 80h ; Вызов ядра
;----- системный вызов 'read' -----
; После вызова инструкции 'int 80h' программа будет ожидать ввода
; строки, которая будет записана в переменную 'buf1' размером 80 байт
mov eax,3 ; Системный вызов для чтения (sys_read)
```

Рисунок 2.7: Редактирование программы в mscedit

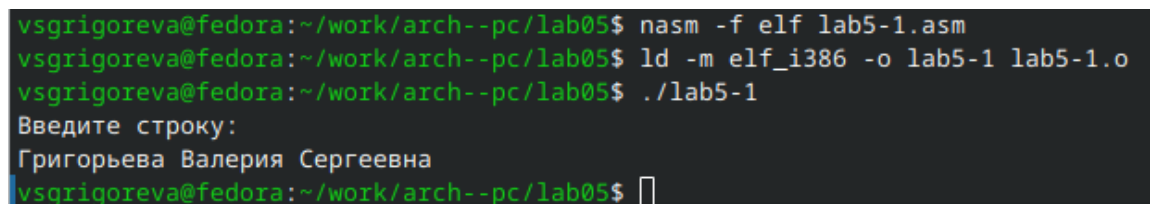
Затем я сохранила файл с помощью клавиши F2 и вышла из редактора с помощью F10. Чтобы посмотреть файл и убедиться, что файл содержит текст программы, я нажала клавишу F3 (Рисунок 2.8).



```
/home/vsgrigoreva/work/arch--pc/lab05/lab5-1.asm 1828/2431 75%
;-----
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----
;----- Объявление переменных -----
SECTION .data ; Секция инициализированных данных
msg: DB 'Введите строку:',10 ; сообщение плюс
; символ перевода строки
msgLen: EQU $-msg ; Длина переменной 'msg'
SECTION .bss ; Секция не инициализированных данных
buf1: RESB 80 ; Буфер размером 80 байт
;----- Текст программы -----
SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
;----- Системный вызов 'write'
; После вызова инструкции 'int 80h' на экран будет
; выведено сообщение из переменной 'msg' длиной 'msgLen'
mov eax,4 ; Системный вызов для записи (sys_write)
mov ebx,1 ; Описатель файла 1 - стандартный вывод
mov ecx,msg ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen ; Размер строки 'msg' в 'edx'
int 80h ; Вызов ядра
;----- системный вызов 'read' -----
; После вызова инструкции 'int 80h' программа будет ожидать ввода
; строки, которая будет записана в переменную 'buf1' размером 80 байт
mov eax, 3 ; Системный вызов для чтения (sys_read)
```

Рисунок 2.8: Просмотр файла

Затем я оттранслировала текст программы в объектный файл, выполнила компоновку объектного файла и запустила его. Программа вывела строку „Введите строку:“, я ввела ФИО (Рисунок 2.9). Программа корректно завершила работу.



```
vsgrigoreva@fedora:~/work/arch--pc/lab05$ nasm -f elf lab5-1.asm
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ld -m elf_i386 -o lab5-1 lab5-1.o
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ./lab5-1
Введите строку:
Григорьева Валерия Сергеевна
vsgrigoreva@fedora:~/work/arch--pc/lab05$
```

Рисунок 2.9: Запуск программы из файла

Для упрощения написания программ часто встречающиеся одинаковые участки кода (такие как, например, вывод строки на экран или выход из программы) можно оформить в виде подпрограмм и сохранить в отдельные файлы, а во всех нужных местах поставить вызов нужной подпрограммы. NASM позволяет подключать внешние файлы с помощью директивы `%include`, которая предписывает ассемблеру заменить

эту директиву содержимым файла. Для выполнения лабораторных работ используется файл `in_out.asm`. Для начала работы я скачала его со страницы курса в ТУИС. Так как подключаемый файл должен находиться в том же каталоге, что и файл с программой, в которой он используется, необходимо было переместить его. С помощью клавиши F5 я скопировала файл (Рисунок 2.10).

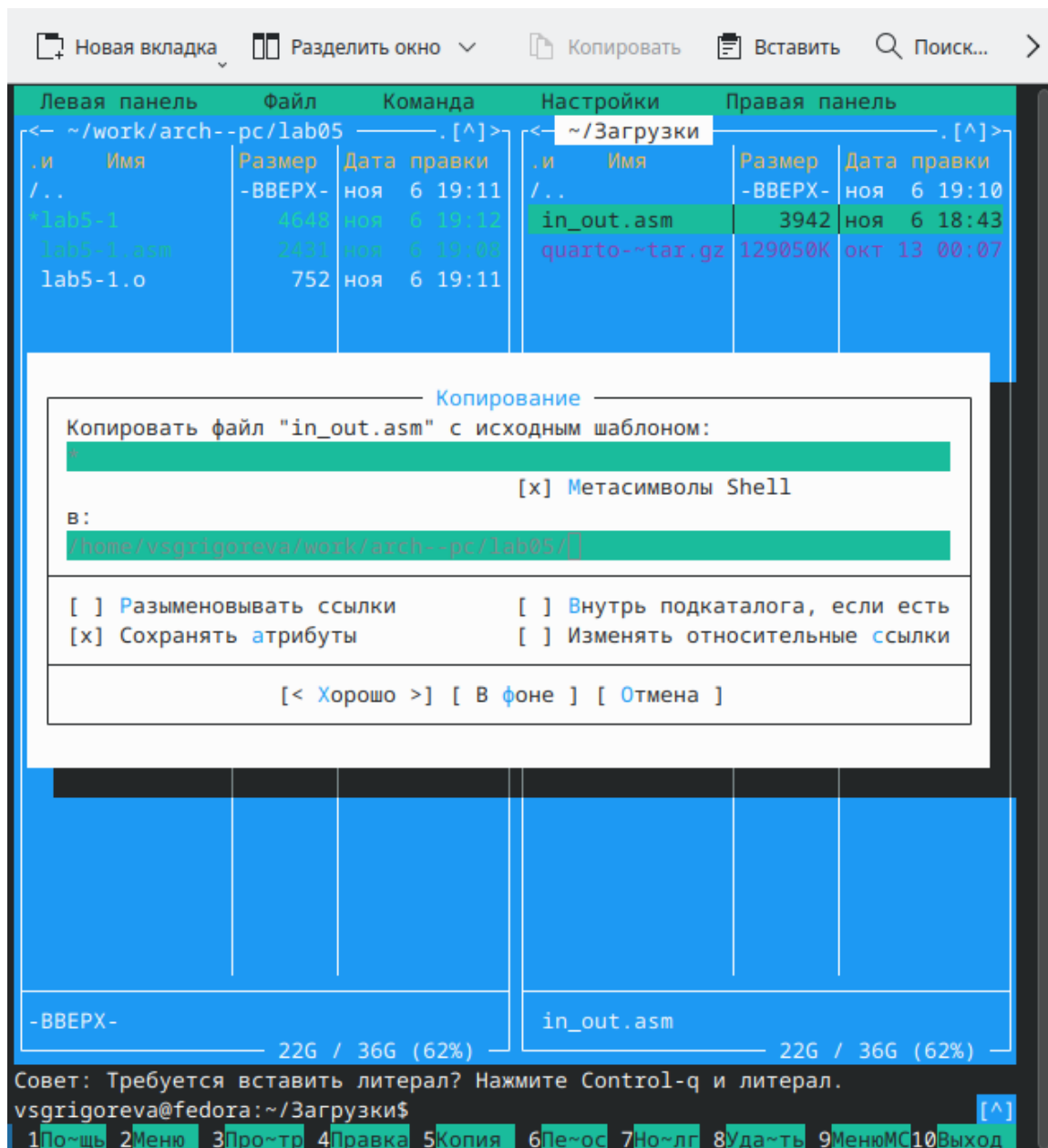
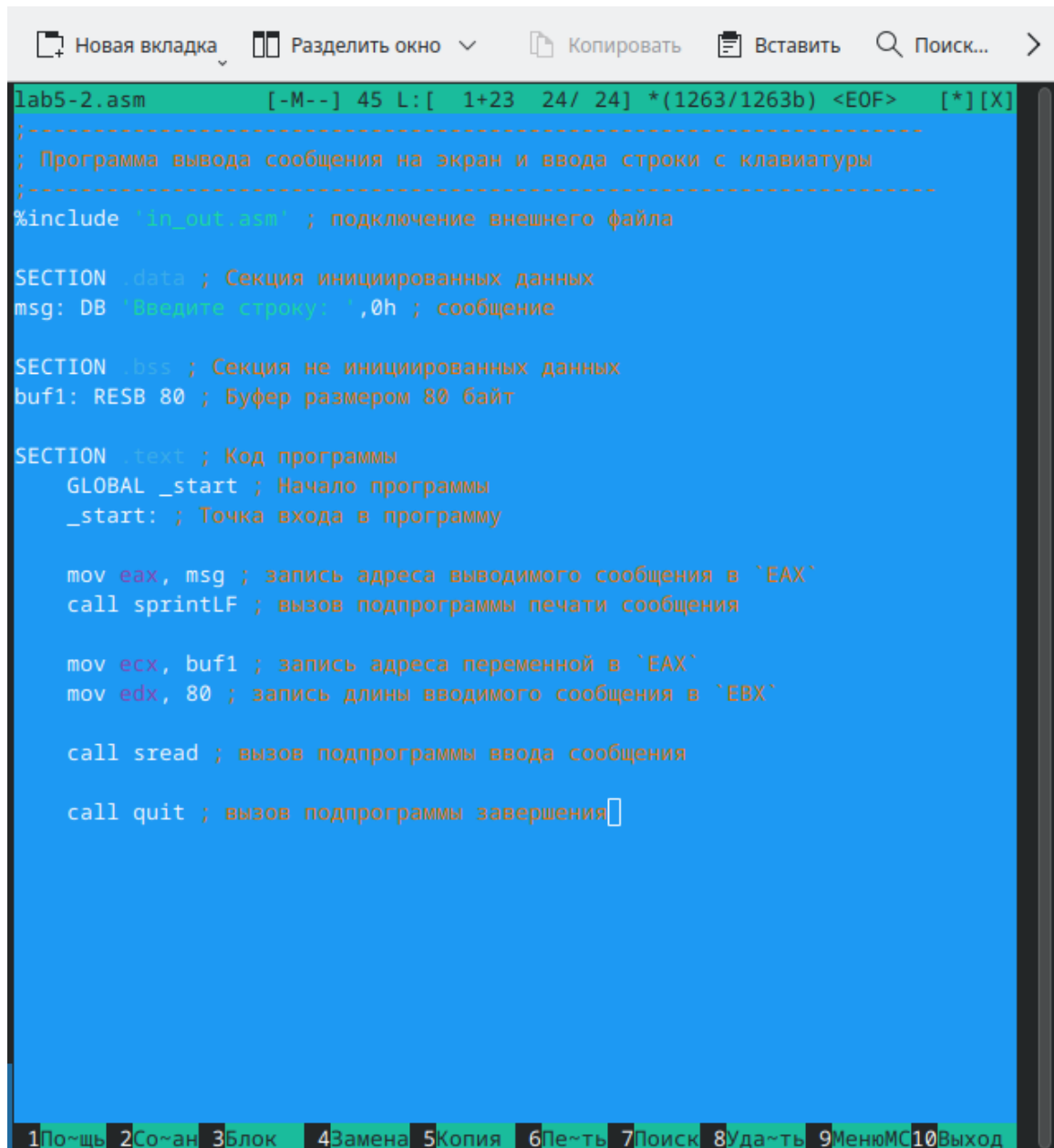


Рисунок 2.10: Копирование файла

С помощью клавиши F6 создала копию файла lab5-1.asm с именем lab5-2.asm. Затем я исправила текст программы в файле lab5-2.asm с использованием подпрограмм из внешнего файла in_out.asm (Рисунок 2.11).



```
lab5-2.asm [-M--] 45 L:[ 1+23 24/ 24] *(1263/1263b) <EOF> [*][X]
;-----
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----
#include 'in_out.asm' ; подключение внешнего файла

SECTION .data ; Секция инициированных данных
msg: DB 'Введите строку: ',0h ; сообщение

SECTION .bss ; Секция не инициированных данных
buf1: RESB 80 ; Буфер размером 80 байт

SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу

mov eax, msg ; запись адреса выводимого сообщения в 'EAX'
call sprintf ; вызов подпрограммы печати сообщения

mov ecx, buf1 ; запись адреса переменной в 'EAX'
mov edx, 80 ; запись длины вводимого сообщения в 'EBX'

call sread ; вызов подпрограммы ввода сообщения

call quit ; вызов подпрограммы завершения
```

Рисунок 2.11: Редактирование файла lab5-2.asm

Чтобы посмотреть файл и убедиться, что он содержит текст программы, я нажала клавишу F3 (Рисунок 2.12).

```
/home/vsgrigoreva/work-pc/lab05/lab5-2.asm 1263/1263 100%
;-----
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----
%include 'in_out.asm' ; подключение внешнего файла

SECTION .data ; Секция инициированных данных
msg: DB 'Введите строку: ',0h ; сообщение

SECTION .bss ; Секция не инициированных данных
buf1: RESB 80 ; Буфер размером 80 байт

SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу

    mov eax, msg ; запись адреса выводимого сообщения в `EAX`
    call sprintf ; вызов подпрограммы печати сообщения

    mov ecx, buf1 ; запись адреса переменной в `EAX`
    mov edx, 80 ; запись длины вводимого сообщения в `EBX`

    call sread ; вызов подпрограммы ввода сообщения

    call quit ; вызов подпрограммы завершения
```

1По~щъ 2Ра~рн 3Выход 4Нех 5Пер~ти 6 7Поиск 8Исх~ый 9Формат10Выход

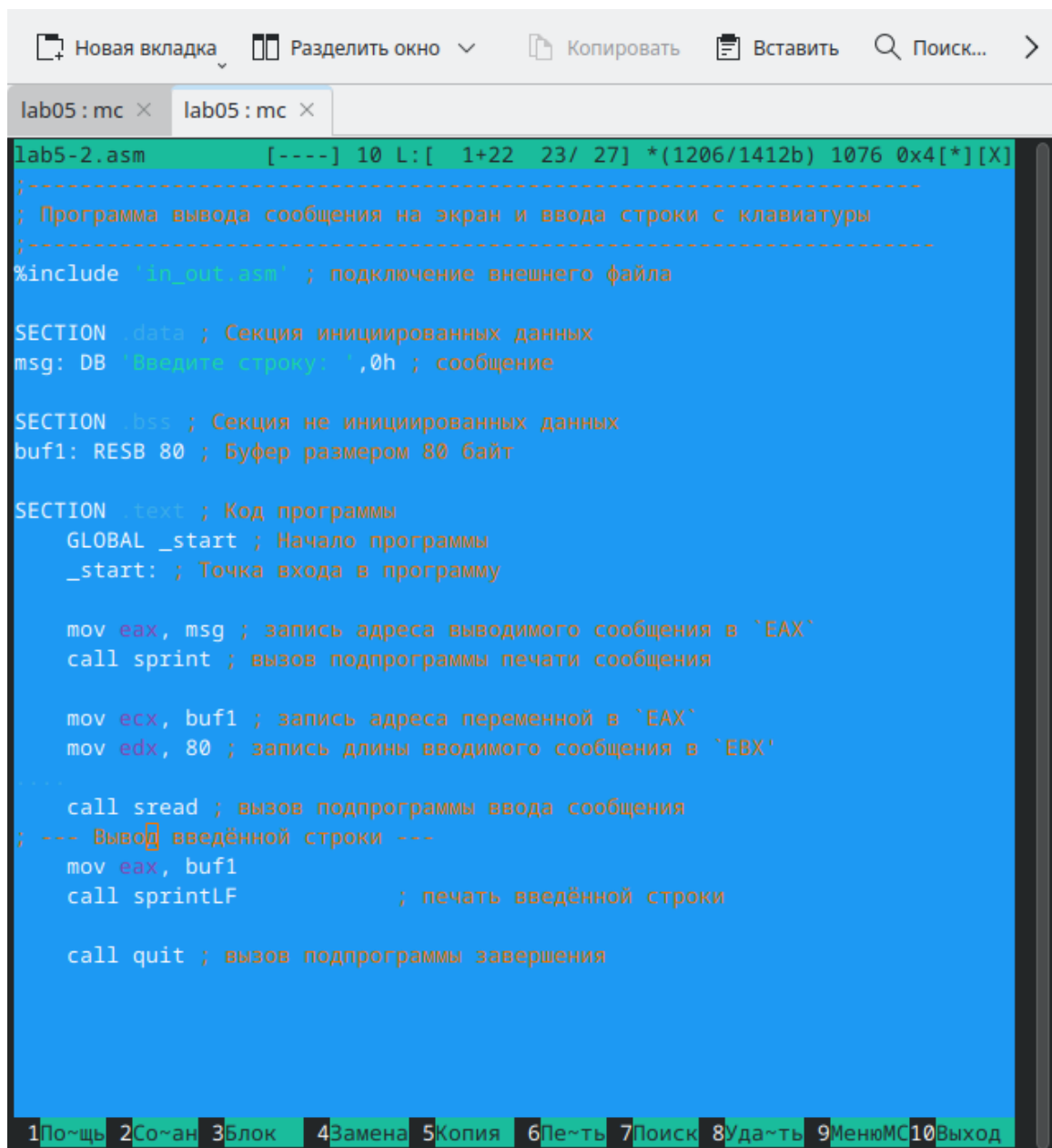
Рисунок 2.12: Просмотр файла

Затем я оттранслировала текст программы в объектный файл, выполнила компоновку объектного файла и запустила его. Программа вывела строку „Введите строку:“, я ввела ФИО (Рисунок 2.13). Программа корректно завершила работу.

```
vsgrigoreva@fedora:~/work/arch--pc/lab05$ nasm -f elf lab5-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ld -m elf_i386 -o lab5-2 lab5-2.o
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ./lab5-2
Введите строку:
Григорьева Валерия Сергеевна
vsgrigoreva@fedora:~/work/arch--pc/lab05$
```

Рисунок 2.13: Запуск программы из файла

В файле lab5-2.asm заменила подпрограмму sprintLF на sprint (Рисунок 2.14).



```
lab5-2.asm [----] 10 L: [ 1+22 23/ 27] *(1206/1412b) 1076 0x4[*][X]
;
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;
#include 'in_out.asm' ; подключение внешнего файла

SECTION .data ; Секция инициализированных данных
msg: DB 'Введите строку: ',0h ; сообщение

SECTION .bss ; Секция не инициализированных данных
buf1: RESB 80 ; Буфер размером 80 байт

SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу

    mov eax, msg ; запись адреса выводимого сообщения в 'EAX'
    call sprint ; вызов подпрограммы печати сообщения

    mov ecx, buf1 ; запись адреса переменной в 'EAX'
    mov edx, 80 ; запись длины вводимого сообщения в 'EBX'

    call sread ; вызов подпрограммы ввода сообщения
; --- Вывод введенной строки ---
    mov eax, buf1
    call sprintf ; печать введенной строки

    call quit ; вызов подпрограммы завершения
```

1Поиск 2Сохранить 3Блок 4Замена 5Копия 6Печать 7Поиск 8Удалить 9Меню 10Выход

Рисунок 2.14: Замена подпрограммы

Затем я оттранслировала его в объектный файл и запустила его. Программа вывела строку „Введите строку:“, я ввела ФИО (Рисунок 2.15). Программа корректно завершила работу.

```
vsgrigoreva@fedora:~/work/arch--pc/lab05$ nasm -f elf lab5-2.asm
vsgrigoreva@fedora:~/work/arch--pc/lab05$ d -m elf_i386 -o lab5-2 lab5-2.o
bash: d: команда не найдена...
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ld -m elf_i386 -o lab5-2 lab5-2.o
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ./lab5-2
Введите строку: Григорьева Валерия Сергеевна
Григорьева Валерия Сергеевна

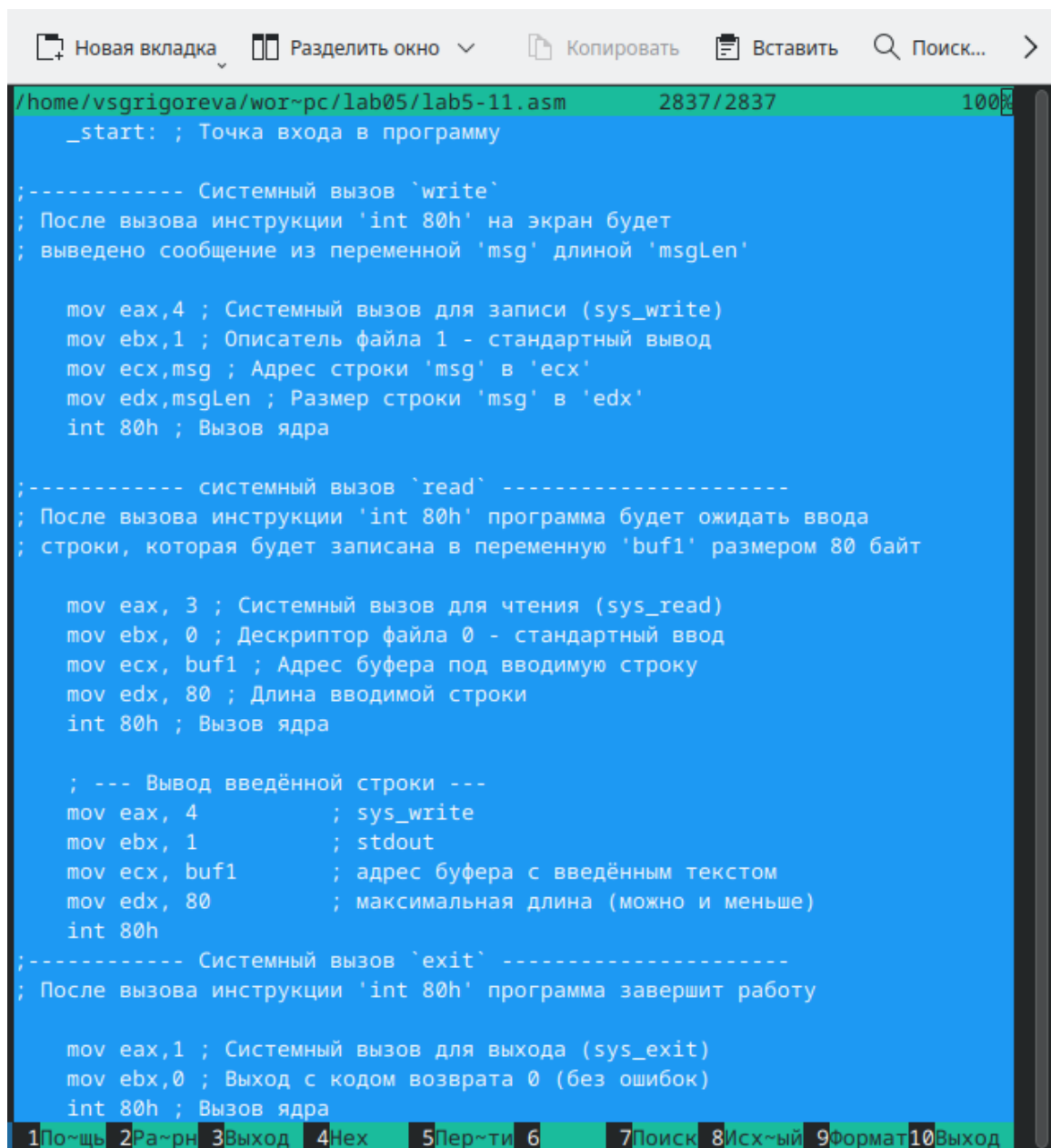
vsgrigoreva@fedora:~/work/arch--pc/lab05$
```

Рисунок 2.15: Запуск программы из файла

При вводе команды `sprintLF` в текст программы, происходит перенос строки на следующую, а при `sprint` программа предлагает ввести текст на той же строке, на которой выводится запрос.

3 Задание для самостоятельной работы

1, 2. Я создала с помощью клавиши F6 копию файла lab5-1.asm, которую назвала lab5-11.asm. Затем я внесла изменения в программу таким образом, чтобы она выводила строку, которую я ввела (Рисунок 3.1).



```
/home/vsgrigoreva/work/pc/lab05/lab5-11.asm 2837/2837 100%
_start: ; Точка входа в программу

;----- Системный вызов `write`
; После вызова инструкции `int 80h` на экран будет
; выведено сообщение из переменной `msg` длиной `msgLen`

    mov eax,4 ; Системный вызов для записи (sys_write)
    mov ebx,1 ; Описатель файла 1 - стандартный вывод
    mov ecx,msg ; Адрес строки `msg` в `ecx`
    mov edx,msgLen ; Размер строки `msg` в `edx`
    int 80h ; Вызов ядра

;----- системный вызов `read` -----
; После вызова инструкции `int 80h` программа будет ожидать ввода
; строки, которая будет записана в переменную `buf1` размером 80 байт

    mov eax, 3 ; Системный вызов для чтения (sys_read)
    mov ebx, 0 ; Дескриптор файла 0 - стандартный ввод
    mov ecx, buf1 ; Адрес буфера под вводимую строку
    mov edx, 80 ; Длина вводимой строки
    int 80h ; Вызов ядра

; --- Вывод введенной строки ---
    mov eax, 4 ; sys_write
    mov ebx, 1 ; stdout
    mov ecx, buf1 ; адрес буфера с введенным текстом
    mov edx, 80 ; максимальная длина (можно и меньше)
    int 80h

;----- Системный вызов `exit` -----
; После вызова инструкции `int 80h` программа завершит работу

    mov eax,1 ; Системный вызов для выхода (sys_exit)
    mov ebx,0 ; Выход с кодом возврата 0 (без ошибок)
    int 80h ; Вызов ядра
```

Рисунок 3.1: Просмотр программы файла lab5-11.asm

В листинге [lst:lab5-11] представлена программа lab5-11.asm, реализующая ввод строки с клавиатуры и её последующий вывод на экран.

```

;-----
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----
;----- Объявление переменных -----
SECTION .data ; Секция иницированных данных
msg: DB 'Введите строку:',10 ; сообщение плюс
; символ перевода строки
msgLen: EQU $-msg ; Длина переменной 'msg'
SECTION .bss ; Секция не иницированных данных
buf1: RESB 80 ; Буфер размером 80 байт
;----- Текст программы -----
SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
;----- Системный вызов `write`
; После вызова инструкции 'int 80h' на экран будет
; выведено сообщение из переменной 'msg' длиной 'msgLen'
mov eax,4 ; Системный вызов для записи (sys_write)
mov ebx,1 ; Описатель файла 1 - стандартный вывод
mov ecx,msg ; Адрес строки 'msg' в 'ecx'
mov edx,msgLen ; Размер строки 'msg' в 'edx'
int 80h ; Вызов ядра
;----- системный вызов `read` -----
; После вызова инструкции 'int 80h' программа будет ожидать ввода
; строки, которая будет записана в переменную 'buf1' размером 80 байт
mov eax, 3 ; Системный вызов для чтения (sys_read)
mov ebx, 0 ; Дескриптор файла 0 - стандартный ввод
mov ecx, buf1 ; Адрес буфера под вводимую строку

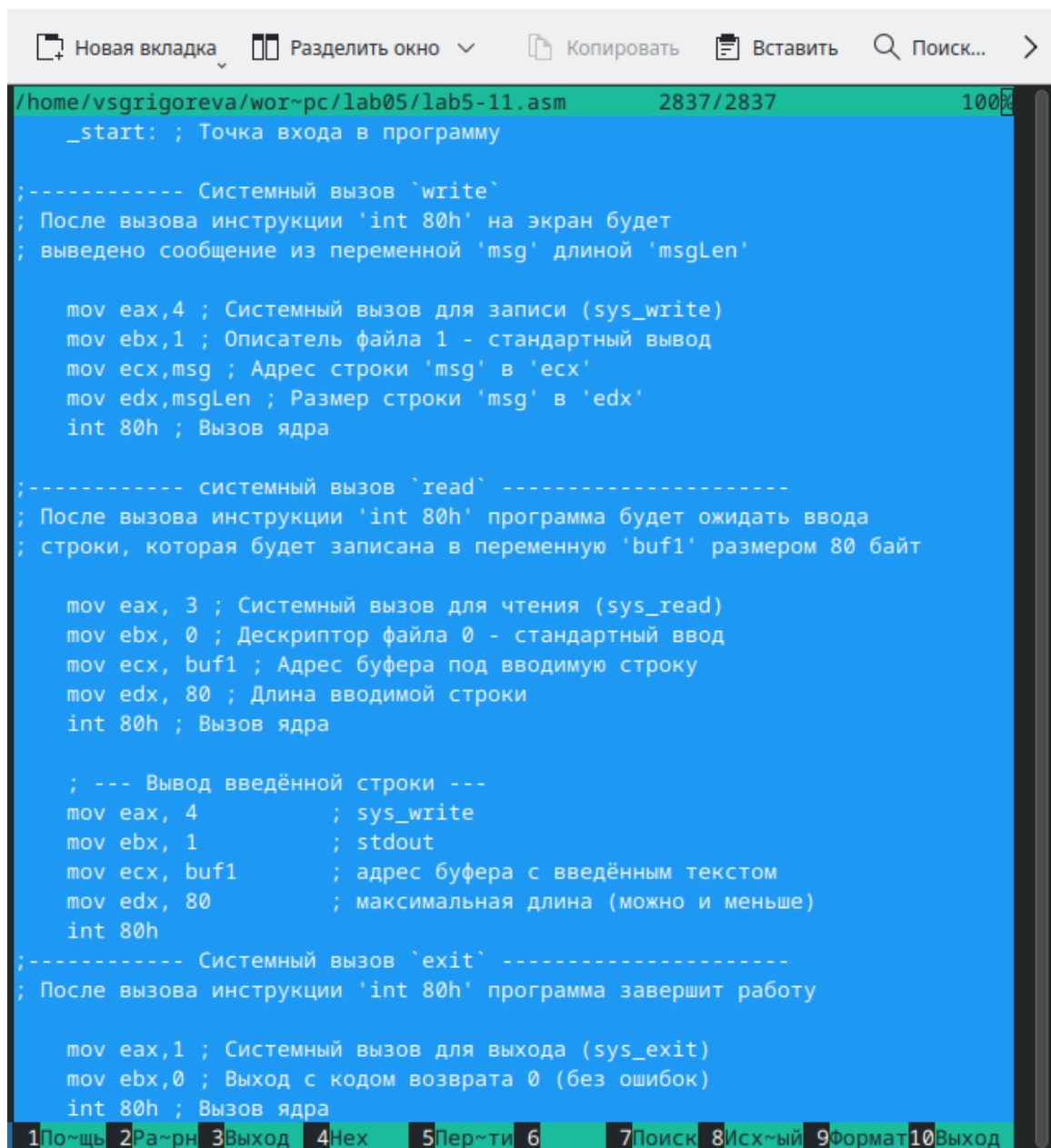
```

```

mov edx, 80 ; Длина вводимой строки
int 80h ; Вызов ядра
; --- Вывод введённой строки ---
mov eax, 4 ; sys_write
mov ebx, 1 ; stdout
mov ecx, buf1 ; адрес буфера с введённым текстом
mov edx, 80 ; максимальная длина (можно и меньше)
int 80h
;----- Системный вызов `exit` -----
; После вызова инструкции 'int 80h' программа завершит работу
mov eax,1 ; Системный вызов для выхода (sys_exit)
mov ebx,0 ; Выход с кодом возврата 0 (без ошибок)
int 80h ; Вызов ядра

```

Далее я получила исполняемый файл и проверила его работу (Рисунок 3.2). Программа вывела введенную мной строку и корректно завершила работу.



```
/home/vsgrigoreva/work/pc/lab05/lab5-11.asm 2837/2837 100%
_start: ; Точка входа в программу

;----- Системный вызов `write`
; После вызова инструкции `int 80h` на экран будет
; выведено сообщение из переменной `msg` длиной `msgLen`

    mov eax,4 ; Системный вызов для записи (sys_write)
    mov ebx,1 ; Описатель файла 1 - стандартный вывод
    mov ecx,msg ; Адрес строки `msg` в `ecx`
    mov edx,msgLen ; Размер строки `msg` в `edx`
    int 80h ; Вызов ядра

;----- системный вызов `read` -----
; После вызова инструкции `int 80h` программа будет ожидать ввода
; строки, которая будет записана в переменную `buf1` размером 80 байт

    mov eax, 3 ; Системный вызов для чтения (sys_read)
    mov ebx, 0 ; Дескриптор файла 0 - стандартный ввод
    mov ecx, buf1 ; Адрес буфера под вводимую строку
    mov edx, 80 ; Длина вводимой строки
    int 80h ; Вызов ядра

; --- Вывод введенной строки ---
    mov eax, 4 ; sys_write
    mov ebx, 1 ; stdout
    mov ecx, buf1 ; адрес буфера с введенным текстом
    mov edx, 80 ; максимальная длина (можно и меньше)
    int 80h

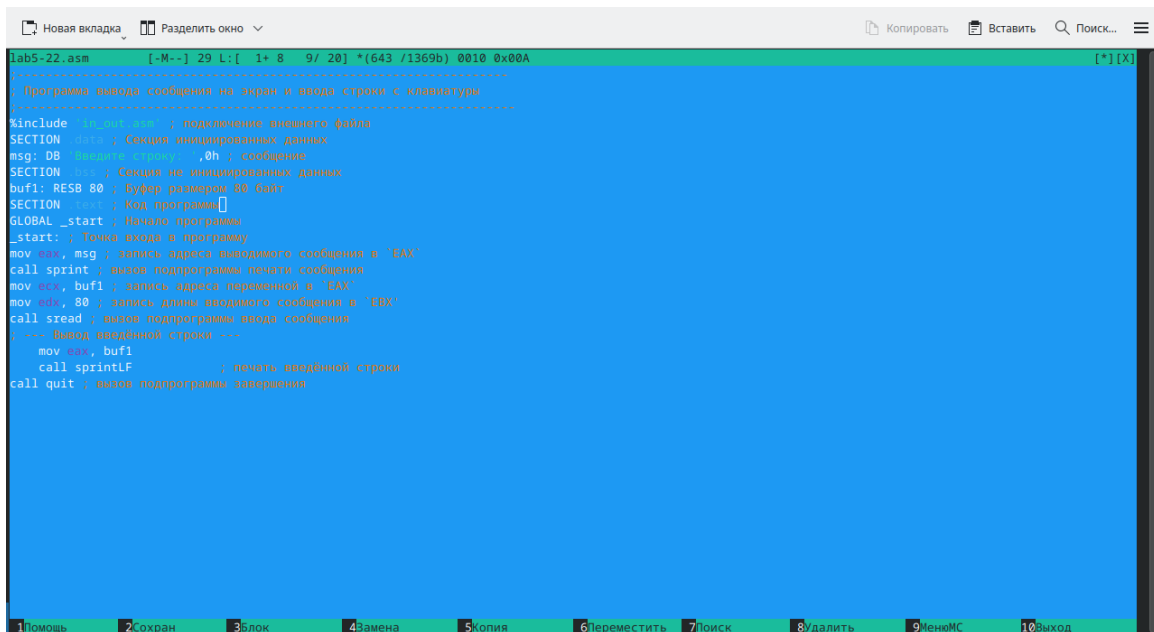
;----- Системный вызов `exit` -----
; После вызова инструкции `int 80h` программа завершит работу

    mov eax,1 ; Системный вызов для выхода (sys_exit)
    mov ebx,0 ; Выход с кодом возврата 0 (без ошибок)
    int 80h ; Вызов ядра

1По~щ 2Ра~рн 3Выход 4Нех 5Пер~ти 6 7Поиск 8Исх~ый 9Формат10Выход
```

Рисунок 3.2: Запуск программы из файла lab5-11.asm

3, 4. С помощью клавиши F6 я создала копию файла lab5-2.asm, которую назвала lab5-22.asm, и в ней дописала текст таким образом, чтобы программа выводила введенную строку на экран (Рисунок 3.3).



```
lab5-22.asm [-M--] 29 L:[ 1+ 8 9/ 20] *(643 /1369b) 0010 0x00A [*][X]

;-----
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----

%include 'in_out.asm' ; подключение внешнего файла
SECTION .data ; Секция иницированных данных
msg: DB 'Введите строку: ',0h ; сообщение
SECTION .bss ; Секция не иницированных данных
buf1: RESB 80 ; Буфер размером 80 байт
SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
mov eax, msg ; запись адреса выводимого сообщения в `EAX`
call sprint ; вызов подпрограммы печати сообщения
mov ecx, buf1 ; запись адреса переменной в `ECX`
mov edx, 80 ; запись длины выводимого сообщения в `EDX`
call sread ; вызов подпрограммы ввода сообщения
; --- Вывод введенной строки ---
mov ecx, buf1
call sprintf ; печать введенной строки
call quit ; вызов подпрограммы завершения

1|Помощь 2|Сохран 3|Закл 4|Бамена 5|Копия 6|Переместить 7|Поиск 8|Удалить 9|МенюМС 10|Выход
```

Рисунок 3.3: Содержимое файла lab5-22.asm

В листинге [lst:lab5-22] представлена программа lab5-22.asm, реализующая ввод строки с клавиатуры и её последующий вывод на экран.

```
;-----
; Программа вывода сообщения на экран и ввода строки с клавиатуры
;-----

%include 'in_out.asm' ; подключение внешнего файла
SECTION .data ; Секция иницированных данных
msg: DB 'Введите строку: ',0h ; сообщение
SECTION .bss ; Секция не иницированных данных
buf1: RESB 80 ; Буфер размером 80 байт
SECTION .text ; Код программы
GLOBAL _start ; Начало программы
_start: ; Точка входа в программу
mov eax, msg ; запись адреса выводимого сообщения в `EAX`
call sprint ; вызов подпрограммы печати сообщения
```

```

mov ecx, buf1 ; запись адреса переменной в `EAX`
mov edx, 80 ; запись длины вводимого сообщения в `EBX`
call sread ; вызов подпрограммы ввода сообщения
; --- Вывод введенной строки ---
    mov eax, buf1
    call sprintLF ; печать введенной строки
call quit ; вызов подпрограммы завершения

```

Затем я создала исполняемый файл и проверила его работу (Рисунок 3.4). Программа вывела введенную мной строку и корректно завершила работу.

```

vsgrigoreva@fedora:~/work/arch--pc/lab05$ nasm -f elf lab5-22.asm
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ld -m elf_i386 -o lab5-22 lab5-22.o
vsgrigoreva@fedora:~/work/arch--pc/lab05$ ./lab5-22
Введите строку: Григорьева Валерия Сергеевна
Григорьева Валерия Сергеевна
vsgrigoreva@fedora:~/work/arch--pc/lab05$ █

```

Рисунок 3.4: Запуск программы из файла lab5-22.asm

4 Вывод

В ходе лабораторной работы я приобрела практические навыки работы в Midnight Commander, освоила инструкции языка ассемблера `mov` и `int`.